

Programming

High-Level Programming Languages

- ▶ Designed to be more human-readable

High-Level Programming Languages

- ▶ Designed to be more human-readable
- ▶ Easier to work with than low-level languages such as assembly or machine code

High-Level Programming Languages

- ▶ Designed to be more human-readable
- ▶ Easier to work with than low-level languages such as assembly or machine code
- ▶ Examples: C, C++, Java, Python, R, ...

High-Level Programming Languages

- ▶ Designed to be more human-readable
- ▶ Easier to work with than low-level languages such as assembly or machine code
- ▶ Examples: C, C++, Java, Python, R, ...
- ▶ Many more benefits: Testability, Maintainable, Resusable

High-Level Programming Languages

- ▶ Designed to be more human-readable
- ▶ Easier to work with than low-level languages such as assembly or machine code
- ▶ Examples: C, C++, Java, Python, R, ...
- ▶ Many more benefits: Testability, Maintainable, Resusable

High-Level Programming Languages

- ▶ Designed to be more human-readable
- ▶ Easier to work with than low-level languages such as assembly or machine code
- ▶ Examples: C, C++, Java, Python, R, ...
- ▶ Many more benefits: Testability, Maintainable, Resusable

Types of Programming Languages

High-Level Programming Languages

- ▶ Designed to be more human-readable
- ▶ Easier to work with than low-level languages such as assembly or machine code
- ▶ Examples: C, C++, Java, Python, R, ...
- ▶ Many more benefits: Testability, Maintainable, Resusable

Types of Programming Languages

How is a program in high-level language executed on hardware?

Program Execution

Interpretation An interpreter executes the source code line by line, translating each line into machine code and then executing it immediately.

Program Execution

Interpretation An interpreter executes the source code line by line, translating each line into machine code and then executing it immediately.

Compilation The entire source code is translated into machine code (or an intermediate representation like bytecode) by a compiler. This machine code is then directly executed by the hardware.

Program Execution

Interpretation An interpreter executes the source code line by line, translating each line into machine code and then executing it immediately.

Compilation The entire source code is translated into machine code (or an intermediate representation like bytecode) by a compiler. This machine code is then directly executed by the hardware.

Virtual Machines

Program Execution

Interpretation An interpreter executes the source code line by line, translating each line into machine code and then executing it immediately.

Compilation The entire source code is translated into machine code (or an intermediate representation like bytecode) by a compiler. This machine code is then directly executed by the hardware.

Virtual Machines

Hybrid Approaches

Program Execution

Interpretation An interpreter executes the source code line by line, translating each line into machine code and then executing it immediately.

Compilation The entire source code is translated into machine code (or an intermediate representation like bytecode) by a compiler. This machine code is then directly executed by the hardware.

Virtual Machines

Hybrid Approaches

Program Execution

Interpretation An interpreter executes the source code line by line, translating each line into machine code and then executing it immediately.

Compilation The entire source code is translated into machine code (or an intermediate representation like bytecode) by a compiler. This machine code is then directly executed by the hardware.

Virtual Machines

Hybrid Approaches

Each mechanism has its own advantages and disadvantages.

Elements of a Imperative Programming Language

- ▶ Variables

Elements of a Imperative Programming Language

- ▶ Variables
- ▶ Statements

Elements of a Imperative Programming Language

- ▶ Variables
- ▶ Statements
- ▶ Control - IF Condition THEN ... ELSE

Elements of a Imperative Programming Language

- ▶ Variables
- ▶ Statements
- ▶ Control - IF Condition THEN ... ELSE
- ▶ Loop - FOR .. in DO ..., WHILE Condition DO

Elements of a Imperative Programming Language

- ▶ Variables
- ▶ Statements
- ▶ Control - IF Condition THEN ... ELSE
- ▶ Loop - FOR .. in DO ..., WHILE Condition DO
- ▶ Functions

Elements of a Imperative Programming Language

- ▶ Variables
- ▶ Statements
- ▶ Control - IF Condition THEN ... ELSE
- ▶ Loop - FOR .. in DO ..., WHILE Condition DO
- ▶ Functions
- ▶ Modules, Libraries (in multiple files)

Core Python and Control Flow

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language

Types are associated with values (objects), not with variable names

Type checking happens at runtime, not before execution

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:
 - ▶ **Type**

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:
 - ▶ **Type**
 - ▶ **Value**

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:
 - ▶ **Type**
 - ▶ **Value**
 - ▶ **Identity** (address, memory utilization)

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:
 - ▶ **Type**
 - ▶ **Value**
 - ▶ **Identity** (address, memory utilization)
 - ▶ **Scope**

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:
 - ▶ **Type**
 - ▶ **Value**
 - ▶ **Identity** (address, memory utilization)
 - ▶ **Scope**
 - ▶ **Life**

Python Data Types: Overview

- ▶ Python is a **dynamically typed** language
 - Types are associated with values (objects), not with variable names
 - Type checking happens at runtime, not before execution
- ▶ Every value in Python is an **object**
- ▶ Each object has:
 - ▶ **Type**
 - ▶ **Value**
 - ▶ **Identity** (address, memory utilization)
 - ▶ **Scope**
 - ▶ **Life**
- ▶ Built-in data types cover most programming needs

Numeric Data Types

Used for arithmetic and mathematical computations

- ▶ `int` – integers (unbounded precision)
- ▶ `float` – floating-point numbers (IEEE 754)
- ▶ `complex` – complex numbers ($a + bj$)

Examples

```
x = 10
```

```
pi = 3.14
```

```
z = 2 + 3j
```


Boolean Data Type

Used for logical conditions

- ▶ `bool` has only two values:
 - ▶ `True`
 - ▶ `False`
- ▶ `bool` is a subclass of `int`

Example

```
is_valid = True
```

```
True == 1
```

```
False == 0
```


Sequence Data Types

Ordered collections of elements

- ▶ `str` – immutable sequence of characters
- ▶ `list` – mutable sequence
- ▶ `tuple` – immutable sequence
- ▶ `range` – arithmetic progression

Examples

```
s = "hello"  
a = [1, 2, 3]  
t = (1, 2, 3)  
r = range(0, 10, 2)
```


Lists

- ▶ Ordered collection
- ▶ Mutable (can be modified in place)
- ▶ Allows duplicate elements

Example

```
values = [10, 20, 30]  
values.append(40)  
values[0] = 5
```


Tuples

- ▶ Ordered collection
- ▶ Immutable (cannot be modified)
- ▶ Often used for fixed-size records

Example

```
point = (3, 4)  
x, y = point
```

Use case: Coordinates, return values, constant data.

Lists vs Tuples

- ▶ Lists represent *changeable* data
- ▶ Tuples represent *fixed* structure
- ▶ Looping over lists and tuples is similar

Example

```
record = ("Alice", 25)    # tuple: structure fixed  
scores = [80, 85, 90]    # list: values may grow
```

Guideline: If it shouldn't change, use a tuple.

Sets

- ▶ `set` – mutable
- ▶ `frozenset` – immutable
- ▶ Unordered collection
- ▶ No duplicate elements
- ▶ Fast membership testing

Example

```
fs = frozenset([1, 2, 3])  
values = [1, 2, 2, 3, 3]  
unique_values = set(values)
```

Result: `unique_values = {1, 2, 3}`

Set Operations

- ▶ Mathematical set operations supported
- ▶ Very useful in data cleaning

Example

`a = {1, 2, 3}`

`b = {3, 4, 5}`

`a & b` `# intersection`

`a | b` `# union`

`a - b` `# difference`

Dictionaries

- ▶ Mapping from keys to values
- ▶ Keys must be immutable
- ▶ Fast lookup by key

Example

```
counts = {1: 3, 2: 5, 3: 2}  
counts[2] += 1
```

Use case: Frequencies, configuration, labeled data.

Iterating Over Dictionaries

- ▶ Iterate over keys, values, or key–value pairs
- ▶ `keys()`, `values()`, `items()`

Example

```
for face, count in counts.items():  
    print(face, count)
```

Common pattern: Counting and aggregation.

Data Structures and Data Analysis

- ▶ Lists $\rightarrow$ columns of data
- ▶ Dictionaries $\rightarrow$ labeled data
- ▶ Tuples $\rightarrow$ fixed records
- ▶ Sets $\rightarrow$ uniqueness constraints

Preview:

- ▶ NumPy arrays replace lists of numbers
- ▶ Pandas DataFrames replace dictionaries of columns

Binary Data Types

Used for raw binary data and I/O

- ▶ `bytes` – immutable sequence of bytes
- ▶ `bytearray` – mutable sequence of bytes
- ▶ `memoryview` – zero-copy data access

Examples

```
b = b"abc"  
ba = bytearray(b"abc")  
mv = memoryview(b"abc")
```


Special Data Type: None

Represents absence of a value

- ▶ Single value: None
- ▶ Commonly used for:
 - ▶ default arguments
 - ▶ missing values
 - ▶ function return with no value

Example

```
x = None
```


User-Defined Data Types

Created using classes

- ▶ Combine data and behavior
- ▶ Basis of object-oriented programming in Python

Example

```
class Point:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```


Control Flow: Conditional Execution

- ▶ Decision making using `if` / `elif` / `else`
- ▶ Conditions evaluate to `True` or `False`

Example

```
x = 7
```

```
if x < 0:  
    sign = "negative"  
elif x == 0:  
    sign = "zero"  
else:  
    sign = "positive"
```


Control Flow: `for` Loops

- ▶ Used for finite iteration
- ▶ Commonly used with `range` and sequences

Example: Loop over range

```
total = 0
for i in range(1, 6):
    total += i
```

Example: Loop over list

```
values = [2, 4, 6]
for v in values:
    print(v * v)
```


Control Flow: `while` Loops

- ▶ Used when number of iterations is not known in advance
- ▶ Loop continues while condition remains true

Example

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Caution: Ensure the condition eventually becomes false.

Control Flow: Loop Control Statements

- ▶ `break`: exit loop immediately
- ▶ `continue`: skip to next iteration

Example

```
for x in range(10):  
    if x == 5:  
        break  
    if x % 2 == 0:  
        continue  
    print(x)
```

Result:

Control Flow: Loop Control Statements

- ▶ `break`: exit loop immediately
- ▶ `continue`: skip to next iteration

Example

```
for x in range(10):  
    if x == 5:  
        break  
    if x % 2 == 0:  
        continue  
    print(x)
```

Result: Prints odd numbers less than 5.

Numerical Thinking: Counters

- ▶ Counters track frequency or occurrences
- ▶ Extremely common in simulations and data analysis

Example: Counting outcomes

```
counts = [0, 0, 0, 0, 0, 0]

for roll in [1, 3, 6, 2, 3, 1]:
    counts[roll - 1] += 1
```

Interpretation:

Numerical Thinking: Counters

- ▶ Counters track frequency or occurrences
- ▶ Extremely common in simulations and data analysis

Example: Counting outcomes

```
counts = [0, 0, 0, 0, 0, 0]

for roll in [1, 3, 6, 2, 3, 1]:
    counts[roll - 1] += 1
```

Interpretation: `counts[i]` stores how often face `i+1` occurred.

Numerical Thinking: Accumulators

- ▶ Accumulators combine values across iterations
- ▶ Used for sums, products, averages

Example: Sum of values

```
total = 0  
values = [2, 4, 6, 8]
```

```
for v in values:  
    total += v
```

Pattern:

Numerical Thinking: Accumulators

- ▶ Accumulators combine values across iterations
- ▶ Used for sums, products, averages

Example: Sum of values

```
total = 0  
values = [2, 4, 6, 8]
```

```
for v in values:  
    total += v
```

Pattern: Initialize

Numerical Thinking: Accumulators

- ▶ Accumulators combine values across iterations
- ▶ Used for sums, products, averages

Example: Sum of values

```
total = 0  
values = [2, 4, 6, 8]
```

```
for v in values:  
    total += v
```

Pattern: Initialize → update

Numerical Thinking: Accumulators

- ▶ Accumulators combine values across iterations
- ▶ Used for sums, products, averages

Example: Sum of values

```
total = 0  
values = [2, 4, 6, 8]
```

```
for v in values:  
    total += v
```

Pattern: Initialize → update → final result.

Numerical Thinking: Aggregation Patterns

- ▶ Aggregation = reduce many values to one
- ▶ Common aggregates: sum, min, max

Example: Minimum value

```
values = [7, 3, 9, 2]  
minimum = values[0]
```

```
for v in values:  
    if v < minimum:  
        minimum = v
```

Observation: Built-ins like `min()` hide this loop.

Numerical Thinking: Simple Statistics

- ▶ Statistics combine multiple aggregation steps
- ▶ Often written imperatively first

Example: Mean (average)

```
values = [10, 20, 30, 40]
```

```
total = 0
```

```
for v in values:
```

```
    total += v
```

```
mean = total / len(values)
```

Idea: Statistics = aggregation + normalization.

Numerical Thinking: Simulation Example

- ▶ Simulations repeat simple numerical updates
- ▶ Results emerge from many iterations

Example: Random walk (1D)

```
position = 0
```

```
for step in [-1, 1, 1, -1, 1]:  
    position += step
```

Interpretation: Position is an accumulator over steps.

Numerical Thinking: Why This Matters

- ▶ These loops express mathematical intent
- ▶ NumPy replaces them with vectorized operations

Idea:

- ▶ Loops explain *what* computation means
- ▶ NumPy explains *how* to do it efficiently

Functions: Why Functions?

- ▶ Encapsulate repeated logic

Functions: Why Functions?

- ▶ Encapsulate repeated logic
- ▶ Improve readability and reuse

Functions: Why Functions?

- ▶ Encapsulate repeated logic
- ▶ Improve readability and reuse
- ▶ Make code testable and modular

Functions: Why Functions?

- ▶ Encapsulate repeated logic
- ▶ Improve readability and reuse
- ▶ Make code testable and modular

Functions: Why Functions?

- ▶ Encapsulate repeated logic
- ▶ Improve readability and reuse
- ▶ Make code testable and modular

Idea: A function is a named computation.

Function Definition: `def`

- ▶ Functions are defined using `def`
- ▶ Indentation defines the function body

Example

```
def square(x):  
    return x * x
```

Usage: `square(5)` returns 25

Parameters and Return Values

- ▶ Parameters are inputs to a function
- ▶ `return` sends a value back to the caller
- ▶ A function may return multiple values

Example

```
def min_max(values):  
    return min(values), max(values)
```

Usage:

```
lo, hi = min_max([3, 7, 2, 9])
```


Default Arguments

- ▶ Parameters can have default values
- ▶ Defaults are used when arguments are omitted

Example

```
def power(x, exponent=2):  
    return x ** exponent
```

Usage:

```
power(3)          # 9  
power(3, 3)       # 27
```


Important Note on Default Arguments

- ▶ Default values are evaluated *once*
- ▶ Avoid mutable defaults (lists, dicts)

Problematic Example

```
def add_item(item, items=[]):  
    items.append(item)  
    return items
```

Better Pattern

```
def add_item(item, items=None):  
    if items is None:  
        items = []  
    items.append(item)  
    return items
```


Local vs Global Scope

- ▶ Variables inside functions are local
- ▶ Global variables are visible but should be avoided

Example

```
x = 10
```

```
def foo():  
    x = 5  
    return x
```

Result: `foo()` returns 5, global `x` remains 10.

Modifying Global Variables (Discouraged)

- ▶ `global` allows modification of global variables
- ▶ Makes code harder to reason about

Example

```
count = 0
```

```
def increment():  
    global count  
    count += 1
```

Recommendation: Prefer returning values instead.

Functions and Data Analysis

- ▶ Functions encapsulate data-processing steps
- ▶ Used to build pipelines

Example

```
def mean(values):  
    return sum(values) / len(values)
```

Preview: NumPy replaces this with `np.mean()`.

Exercise 1

What is the output of the following program?

```
b = [1, 2, 3]
c = [4, 5]
a = [ b, c]
print(a)
b[0] = 9
print(a)
d = b
b = [10, 11, 12]
print(a)
d[1] = 25
print(a)
```


Exercise 2: Random Walk

- ▶ Implement a function for 1D random walk
- ▶ Store positions in a list
- ▶ Run multiple trials
- ▶ Compute final displacement statistics

Preview: Each run becomes a row in a table.

Exercise 3: Dice Roll Simulation

- ▶ Simulate large number of dice rolls
- ▶ Count frequencies of outcomes
- ▶ Estimate probabilities empirically

Preview: NumPy will replace explicit loops.

Comprehensions: The Big Idea

- ▶ Compact syntax for building collections
- ▶ Express *what* to compute, not *how*
- ▶ Replace many common loop patterns

Idea: Comprehensions = loops + conditions + construction.

List Comprehensions

- ▶ Create a list from an iterable
- ▶ Most commonly used comprehension

Loop version

```
squares = []  
for x in range(5):  
    squares.append(x * x)
```

Comprehension version

```
squares = [x * x for x in range(5)]
```


List Comprehensions with Conditions

- Conditions can filter elements

Loop version

```
evens = []  
for x in range(10):  
    if x % 2 == 0:  
        evens.append(x)
```

Comprehension version

```
evens = [x for x in range(10) if x % 2 == 0]
```


Set Comprehensions

- ▶ Create a set using comprehension syntax
- ▶ Automatically removes duplicates

Example

```
values = [1, 2, 2, 3, 3, 3]  
unique_squares = {x * x for x in values}
```

Result: {1, 4, 9}

Dictionary Comprehensions

- ▶ Build dictionaries concisely
- ▶ Useful for transformations and counting

Example: Value transformation

```
values = [1, 2, 3, 4]  
squares = {x: x * x for x in values}
```

Result: {1:1, 2:4, 3:9, 4:16}

Dictionary Comprehensions with Conditions

- Conditions can filter key–value pairs

Example

```
scores = {"Alice": 85, "Bob": 72, "Eve": 91}
```

```
passed = {name: score  
          for name, score in scores.items()  
          if score >= 80}
```

Result: Only students who passed.

Conditional Expressions (Inline if--else)

- ▶ Choose between two values inside an expression
- ▶ Often used inside comprehensions

Example

```
labels = ["even" if x % 2 == 0 else "odd"  
          for x in range(5)]
```

Result: ["even", "odd", "even", "odd", "even"]

Why Comprehensions Matter for Data Analysis

- ▶ Similar to column-wise operations in Pandas
- ▶ Bridge from loops to vectorized thinking

Preview:

- ▶ List comprehension $\rightarrow$ NumPy vector
- ▶ Dict comprehension $\rightarrow$ DataFrame construction

Idiomatic Python: What Does It Mean?

- ▶ Idiomatic Python = writing code the *Python way*
- ▶ Focus on readability and intent
- ▶ Prefer built-ins over manual bookkeeping

`enumerate(): Index + Value`

- ▶ Use when you need both index and value
- ▶ Avoid manual index counters

Non-idiomatic

```
i = 0
for x in values:
    print(i, x)
    i += 1
```

Idiomatic

```
for i, x in enumerate(values):
    print(i, x)
```


enumerate() in Data Context

- ▶ Useful for labeling outputs
- ▶ Common in reporting and logging

Example

```
for i, value in enumerate(results, start=1):  
    print("Trial", i, ":", value)
```

Note: `start=` avoids off-by-one errors.

`zip()`: Parallel Iteration

- ▶ Iterate over multiple sequences together
- ▶ Stops at the shortest sequence

Non-idiomatic

```
for i in range(len(names)):
    print(names[i], scores[i])
```

Idiomatic

```
for name, score in zip(names, scores):
    print(name, score)
```


zip() for Building Dictionaries

- ▶ Common pattern when pairing labels and values

Example

```
names = ["Alice", "Bob", "Eve"]
```

```
scores = [85, 72, 91]
```

```
result = dict(zip(names, scores))
```

Result: {"Alice":85, "Bob":72, "Eve":91}

Truthiness in Python

- ▶ Objects can be used directly in conditions
- ▶ No need to compare with `True` or `False`

Falsy values include:

- ▶ `0`, `0.0`
- ▶ Empty list, tuple, dict, set
- ▶ Empty string
- ▶ `None`

Truthiness: Examples

Non-idiomatic

```
if len(values) > 0:  
    process(values)
```

Idiomatic

```
if values:  
    process(values)
```

Reads as: “If values exist...”

any() and all()

- ▶ Replace loops that check conditions
- ▶ Make intent explicit

Definitions

- ▶ `any(iterable)` $\rightarrow$ True if *at least one* element is True
- ▶ `all(iterable)` $\rightarrow$ True if *all* elements are True

any() Example

Non-idiomatic

```
found = False
for x in values:
    if x < 0:
        found = True
        break
```

Idiomatic

```
found = any(x < 0 for x in values)
```

Meaning: “Is there any negative value?”

all() Example

Non-idiomatic

```
valid = True
for x in values:
    if x <= 0:
        valid = False
        break
```

Idiomatic

```
valid = all(x > 0 for x in values)
```

Meaning: “Are all values positive?”

Preview: Idioms in Pandas

- ▶ `enumerate()` → row indexing
- ▶ `zip()` → column alignment
- ▶ `any()`, `all()` → data validation

File I/O: Why It Matters

- ▶ Real programs read data from files
- ▶ Results are written back to disk
- ▶ File I/O connects Python to the real world

Context Managers: `with open()`

- ▶ Ensures files are properly closed
- ▶ Safer than manual `open()` / `close()`

Example: Reading a text file

```
with open("data.txt", "r") as f:  
    contents = f.read()
```

Benefit: File is closed automatically, even on errors.

Reading Files Line by Line

- ▶ Useful for large files
- ▶ Avoids loading everything into memory

Example

```
with open("data.txt") as f:  
    for line in f:  
        print(line.strip())
```

Common use: Logs, large datasets.

Writing to a File

- ▶ Use mode "w" to write
- ▶ Existing file is overwritten

Example

```
results = ["mean: 10.5", "max: 23"]  
  
with open("summary.txt", "w") as f:  
    for line in results:  
        f.write(line + "\n")
```


CSV Files: Motivation

- ▶ CSV = Comma-Separated Values
- ▶ Common format for tabular data
- ▶ Rows = records, columns = fields

Reading CSV Files

- ▶ Use Python's built-in `csv` module
- ▶ `DictReader` maps columns to values

Example

```
import csv

with open("data.csv") as f:
    reader = csv.DictReader(f)
    rows = list(reader)
```

Result: Each row is a dictionary.

Processing CSV Data

Example: Compute a mean

```
values = []
```

```
for row in rows:
```

```
    values.append(float(row["score"]))
```

```
mean = sum(values) / len(values)
```

Preview: Pandas will do this in one line.

Virtual Environments

- ▶ Why environments matter
- ▶ `venv` vs `conda`
- ▶ Environment isolation

Package Management

- ▶ Installing packages with `pip`
- ▶ Installing packages with `conda`
- ▶ Reproducibility and dependencies

venv vs conda: Key Differences

Aspect	venv	conda
Comes with Python	Yes	No (separate install)
Manages Python version	No	Yes
Manages Python packages	Yes (<code>pip</code>)	Yes
Manages non-Python libs	No	Yes
Compiled numeric libs	No	Yes
Typical use case	Small / pure Python	Data science / scientific
Environment isolation	Lightweight	Full software stack
Learning curve	Low	Moderate

Rule of thumb:

Use `venv` for simple Python projects, `conda` for data science.

Exercise 4: Environment + Data Script

- ▶ Create a conda environment
- ▶ Install NumPy and Pandas
- ▶ Run CSV $\rightarrow$ stats $\rightarrow$ JSON script

Looking Ahead

- ▶ Replace loops with NumPy arrays
- ▶ Replace dicts/lists with Pandas DataFrames
- ▶ Focus on vectorization and tabular data